Main page
- Getting started with STM32L4
  - Getting started with STM32 : STM32 step by step
  - Getting started with STM32 system peripherals
    - Getting started with GPIO
    - Getting started with EXTI
    - Getting started with PWR
    - Getting started with LPBAM
    - Getting started with DMA
    - Getting started with UART
    - Getting started with SPI
    - Getting started with I2C
    - Getting started with I3C
    - Getting started with TIM
    - Getting started with RTC
    - Getting started with WDG
    - Getting started with HRTIM
    - Getting started with DAC
    - Getting started with ADC
    - Getting started with Programmable Logic ArraY
- Getting started with STM32C5

Approved version. Approved on: 16:37, 27 September 2024

Pages Getting started with I2C

Last edited one year ago

Getting started with I2C

This article explains what is I2C is and how to use it through examples.

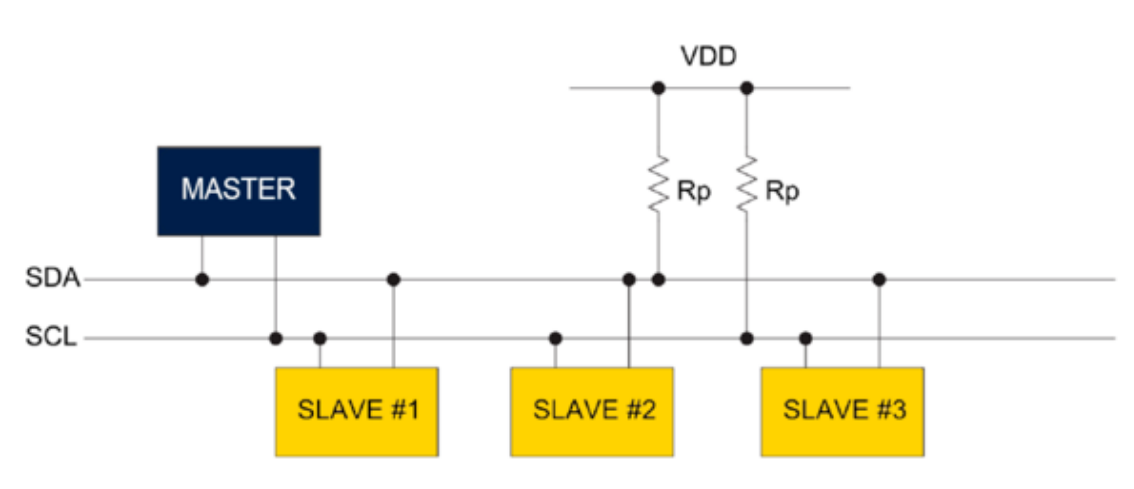
Contents↑ [hide]

1 What is inter-integrated circuit (I2C)?
1.1 I2C Modes
1.2 I2C Frame
1.3 STM32 I2C Mode selection
1.4 Main I2C HAL functions
2 Configure I2C to communicate between two boards
2.1 Objective
2.2 Create project in STM32CubeIDE
2.3 Configure I2C
2.3.1 I2C settings
2.3.2 GPIO Settings
2.3.3 NVIC or DMA settings
2.4 Generate source code and edit main.c
2.4.1 On MASTER Board
2.4.2 On SLAVE Board
2.5 Compile and flash
2.6 Project workspace

1. What is inter-integrated circuit (I2C)?↑

I2C is a two-wire serial communication system used between integrated circuits which was originally created by Philips Semiconductors back in 1982.
The I2C is a multi-master, multi-slave, synchronous, bidirectional, half-duplex serial communication bus.

- ***SDA (Serial Data)*** is the line on which master and slave send or receive the information (sequence of bits).
- ***SCL (Serial Clock)*** is the clock-dedicated line for data flow synchronization.

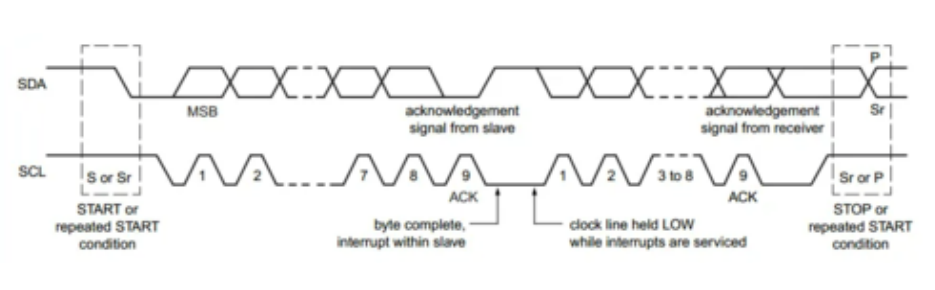

VDD
MASTER
Rp
Rp
SDA
SCL
SLAVE #1
SLAVE #2
SLAVE #3

SDA and SCL lines need to be pulled up with resistors. The value of these resistors depends on the bus length (the bus capacitance) and the transmission speed. The common value is 4.7kΩ. In any case, there are many guides to size them and we refer their reading to the more attentive reader.

1.1. I2C Modes↑

- ***Standard-Mode (Sm)*** with a bit rate up to 100 kbit/s
- ***Fast-Mode (Fm)*** with a bit rate up to 400 kbit/s
- ***Fast-Mode Plus (Fm+)*** with a bit rate up to 1 Mbit/s

1.2. I2C Frame↑


SDA
SCL
MSB
acknowledgement signal from slave
acknowledgement signal from receiver
P
Sr
S or Sr
START or repeated START condition
1
2
7
8
9
ACK
byte complete, interrupt within slave
clock line held LOW while interrupts are serviced
1
2
3 to 8
9
ACK
Sr or P
STOP or repeated START condition

- Start Condition (S)
- Stop Condition (P)
- Repeated Start (Restart) Condition (Sr)
- Acknowledge ACK (A)
- Not Acknowledge NACK (/A)

1.3. STM32 I2C Mode selection↑

The interface can operate in one of the four following modes:

- Slave transmitter
- Slave receiver
- Master transmitter
- Master receiver

1.4. Main I2C HAL functions↑

Based on STM32Cube HAL functions, I2C data transfer can be performed in 3 modes: Blocking Mode, Interrupt Mode or DMA Mode

- **Blocking Mode:**

The communication is performed in polling mode. The status of all data processing is returned by the same function after finishing transfer.

HAL_I2C_Master_Transmit()
HAL_I2C_Master_Receive()
HAL_I2C_Slave_Transmit()
HAL_I2C_Slave_Receive()
HAL_I2C_Mem_Write()
HAL_I2C_Mem_Read()

- **Non-blocking modes**

The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup.
The end of the data processing will be indicated through the dedicated I2C IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.

- **Interrupt Mode:**

HAL_I2C_Master_Transmit_IT()
HAL_I2C_Master_Receive_IT()
HAL_I2C_Slave_Transmit_IT()
HAL_I2C_Slave_Receive_IT()
HAL_I2C_Mem_Write_IT()
HAL_I2C_Mem_Read_IT()

- **DMA Mode:**

HAL_I2C_Master_Transmit_DMA()
HAL_I2C_Master_Receive_DMA()
HAL_I2C_Slave_Transmit_DMA()
HAL_I2C_Slave_Receive_DMA()
HAL_I2C_Mem_Write_DMA()
HAL_I2C_Mem_Read_DMA()

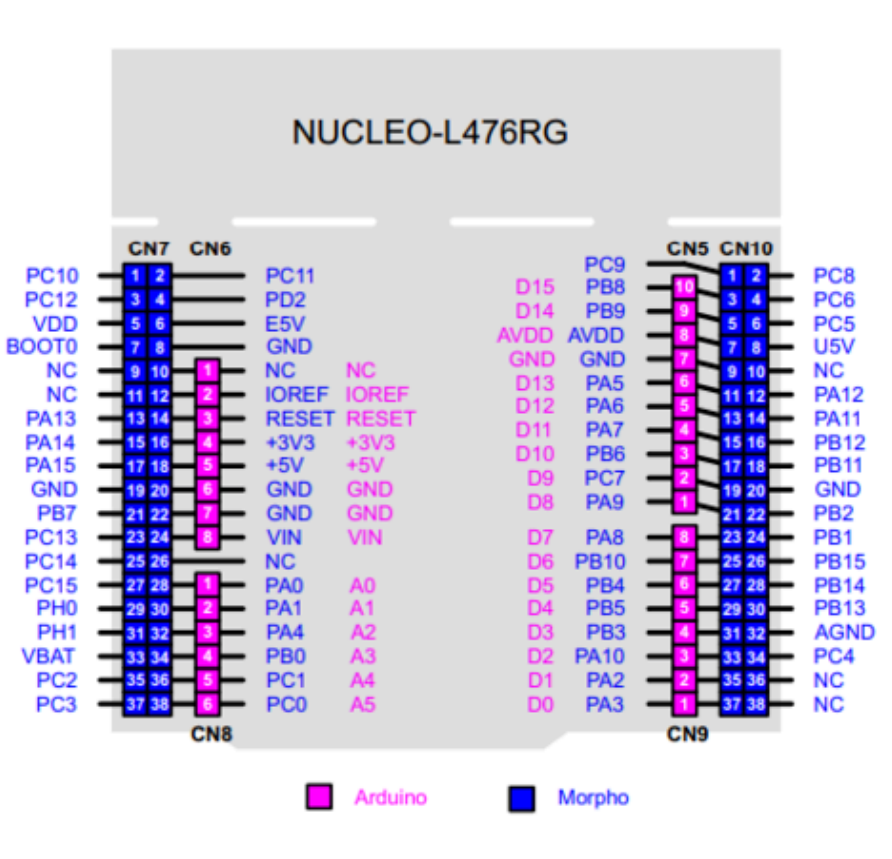
2. Configure I2C to communicate between two boards↑

2.1. Objective↑

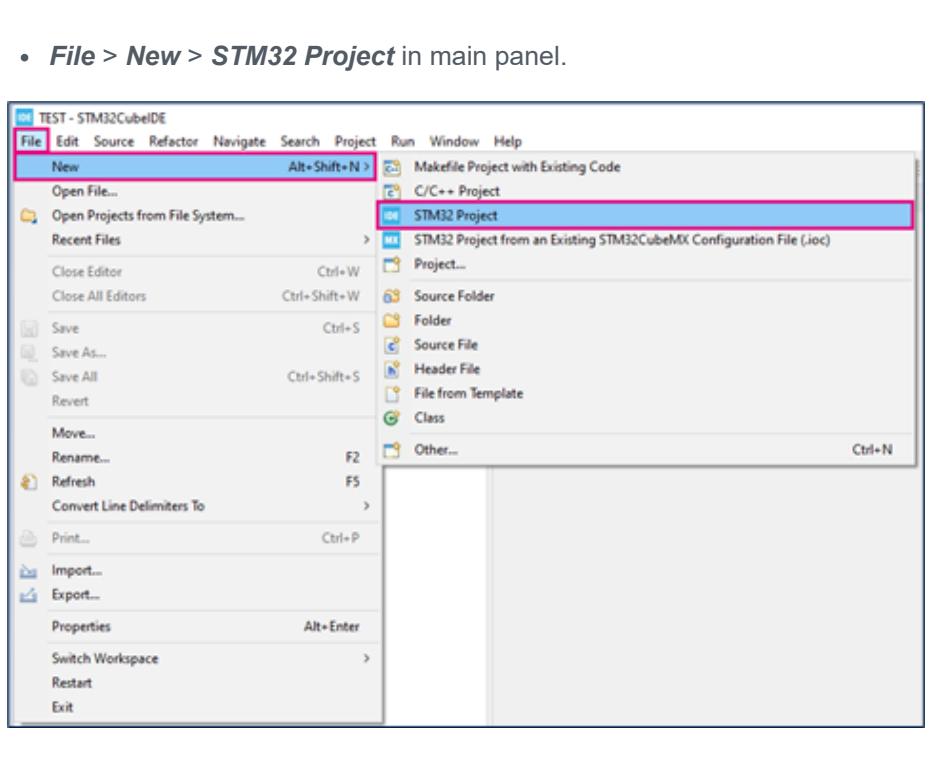
Communicate data between two NUCLEO-L476RG and learn how to set I2C in different modes.

2.2. Create project in STM32CubeIDE↑

- The NUCLEO-L476RG pinout is helpful to set the wiring between the boards in the project.

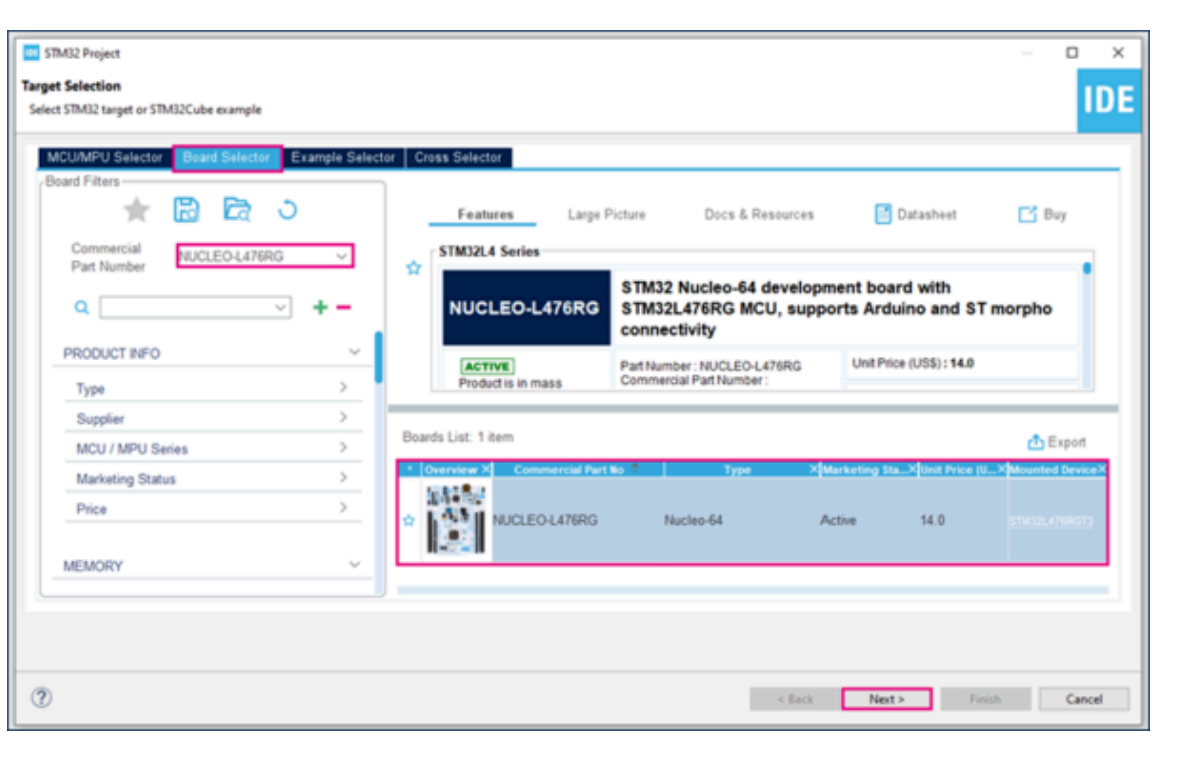

NUCLEO-L476RG
CN7
CN6
CN5
CN10
CN8
CN9
PC10
PC12
VDD
BOOT0
NC
NC
PA13
PA14
PA15
GND
PB7
PC13
PC14
PC15
PH0
PH1
VBAT
PC2
PC3
PC11
PD2
E5V
GND
NC
IOREF
RESET
+3V3
+5V
GND
GND
VIN
NC
PA0
PA1
PA4
PB0
PC1
PC0
A0
A1
A2
A3
A4
A5
D15
D14
AVDD
GND
D13
D12
D11
D10
D9
D8
D7
D6
D5
D4
D3
D2
D1
D0
PC9
PB8
PB9
AVDD
GND
PA5
PA6
PA7
PB6
PC7
PA9
PA8
PB10
PB4
PB5
PB3
PA10
PA2
PA3
PC8
PC6
PC5
U5V
NC
PA12
PA11
PB12
PB11
GND
PB2
PB1
PB15
PB14
PB13
AGND
PC4
NC
NC
Arduino
Morpho

- ***File*** > ***New*** > ***STM32 Project*** in main panel.


TEST - STM32CubeIDE
File
Edit
Source
Refactor
Navigate
Search
Project
Run
Window
Help
New
Alt+Shift+N
Open File...
Open Projects from File System...
Recent Files
Close Editor
Close All Editors
Save
Save As...
Save All
Revert
Move...
Rename...
Refresh
Convert Line Delimiters To
Print...
Import...
Export...
Properties
Switch Workspace
Restart
Exit
Makefile Project with Existing Code
C/C++ Project
STM32 Project
STM32 Project from an Existing STM32CubeMX Configuration File (.ioc)
Project...
Source Folder
Folder
Source File
Header File
File from Template
Class
Other...

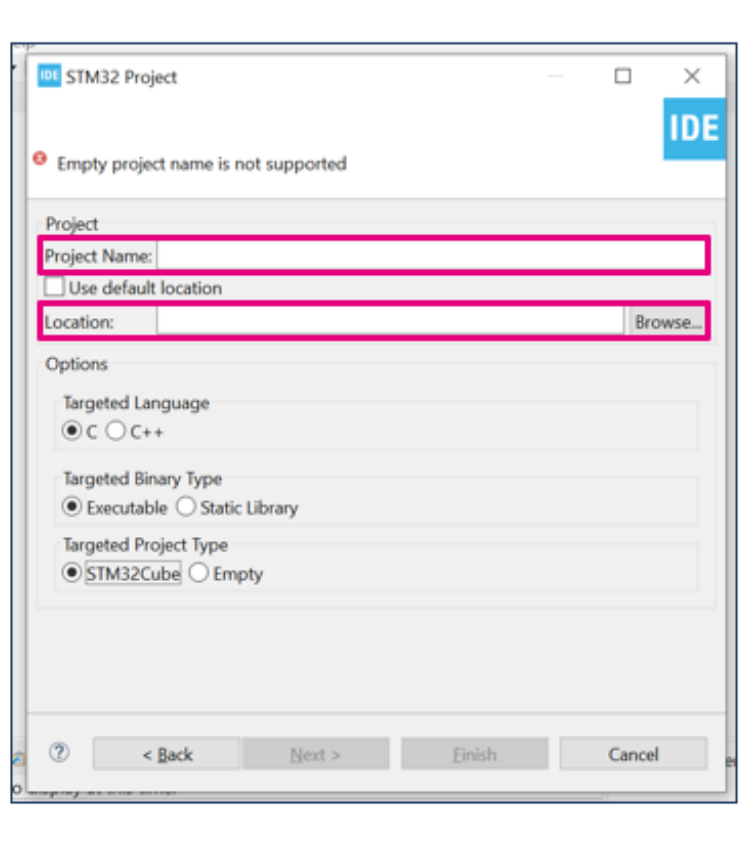
This example uses the NUCLEO-L476RG board.

- Select ***NUCLEO-L476RG*** board using ***Board Selector*** as shown in the figure below.


STM32 Project
Target Selection
Select STM32 target or STM32Cube example
IDE
MCU/MPU Selector
Board Selector
Example Selector
Cross Selector
Board Filters
Commercial Part Number
NUCLEO-L476RG
PRODUCT INFO
Type
Supplier
MCU / MPU Series
Marketing Status
Price
MEMORY
Features
Large Picture
Docs & Resources
Datasheet
Buy
STM32L4 Series
NUCLEO-L476RG
STM32 Nucleo-64 development board with STM32L476RG MCU, supports Arduino and ST morpho connectivity
ACTIVE
Product is in mass
Boards List: 1 item
Export
NUCLEO-L476RG
Nucleo-64
Active
14.0
< Back
Next >
Finish
Cancel

In case you haven't downloaded the STM32L476 Cube library, it will be downloaded automatically. This however may take some time.

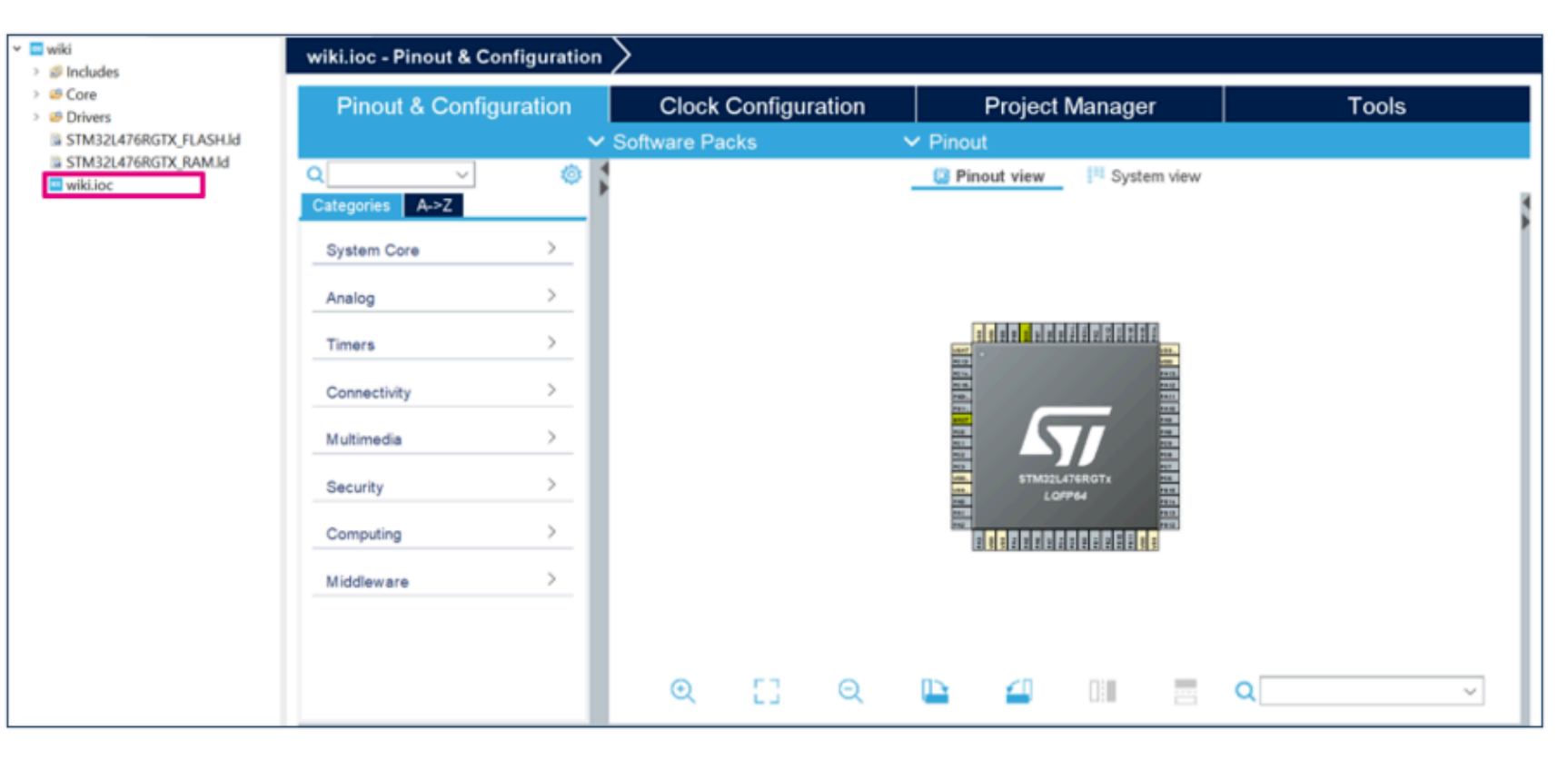
- Save the project.


STM32 Project
IDE
Empty project name is not supported
Project
Project Name:
Use default location
Location:
Browse...
Options
Targeted Language
C
C++
Targeted Binary Type
Executable
Static Library
Targeted Project Type
STM32Cube
Empty
< Back
Next >
Finish
Cancel

2.3. Configure I2C↑

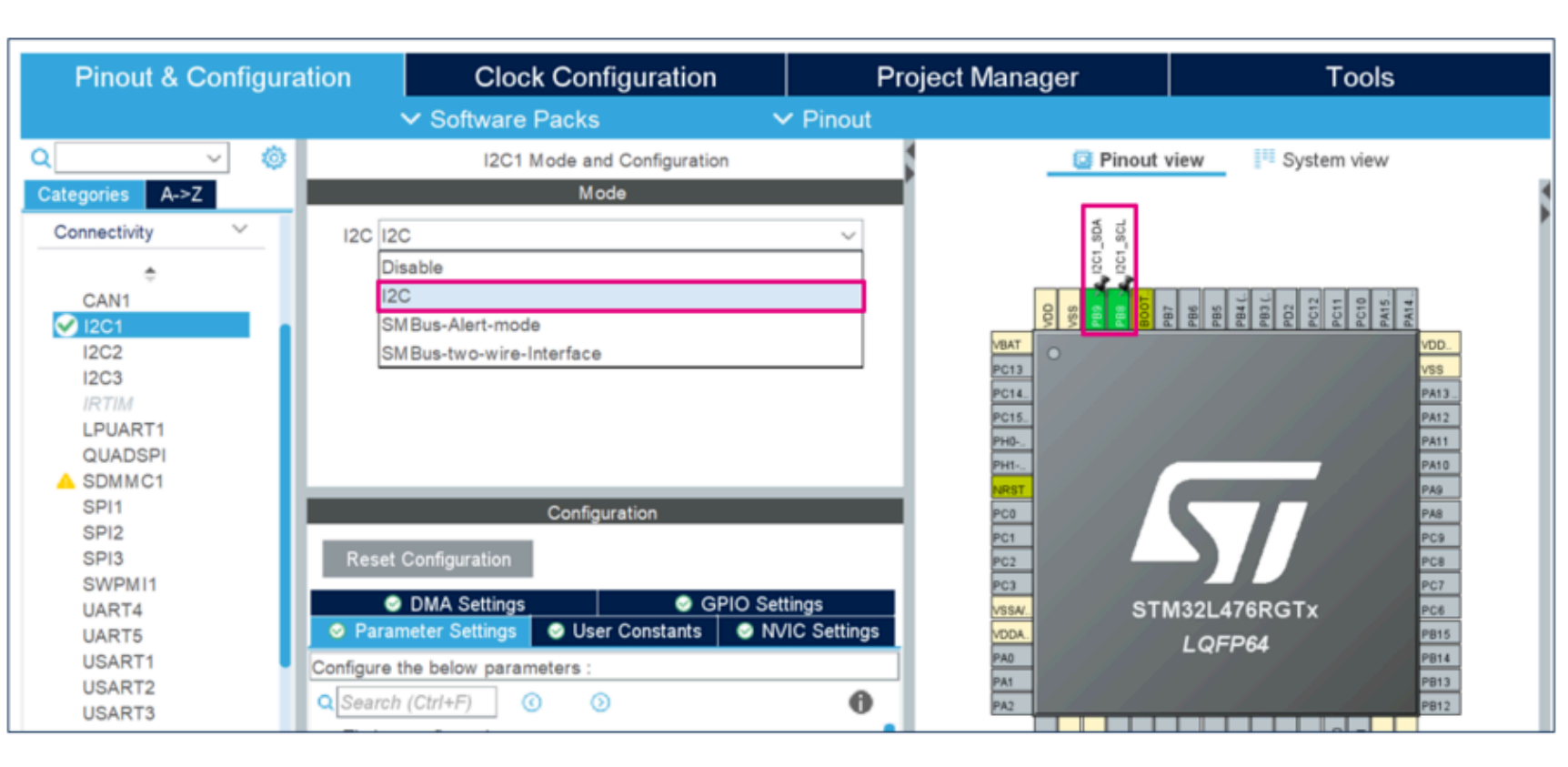
Information
Settings are the same for master and slave boards.

- Open the STM32CubeMX project from the workspace

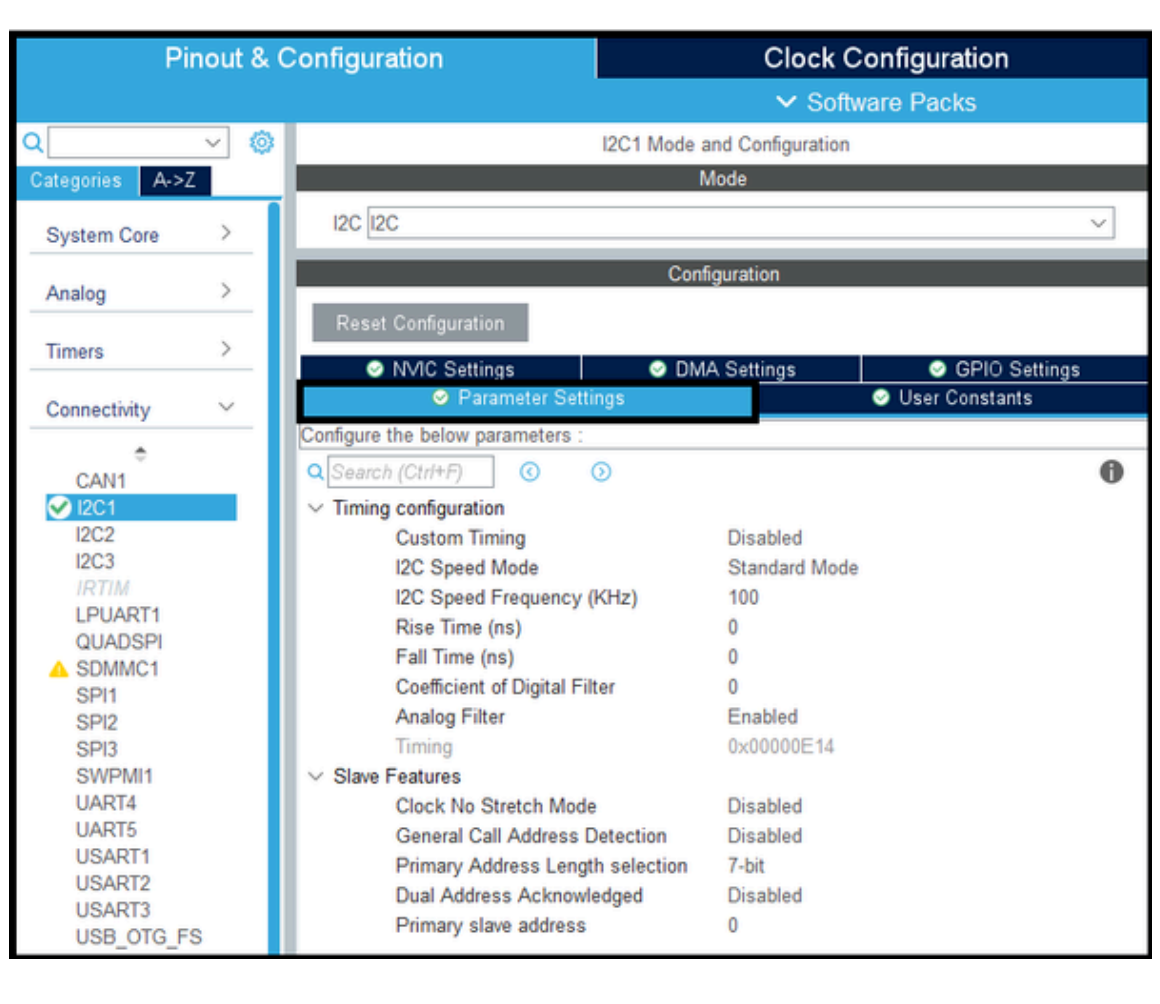

wiki
Includes
Core
Drivers
STM32L476RGTX_FLASH.ld
STM32L476RGTX_RAM.ld
wiki.ioc
wiki.ioc - Pinout & Configuration
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
System Core
Analog
Timers
Connectivity
Multimedia
Security
Computing
Middleware
Pinout view
System view
STM32L476RGTx
LQFP64

2.3.1. I2C settings↑

- Under **Connectivity**, select **I2C1**
- Select **PB8: SCL** and **PB9: SDA** for I2C1


Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
Connectivity
CAN1
I2C1
I2C2
I2C3
IRTIM
LPUART1
QUADSPI
SDMMC1
SPI1
SPI2
SPI3
SWPMI1
UART4
UART5
USART1
USART2
USART3
I2C1 Mode and Configuration
Mode
I2C
Disable
I2C
SMBus-Alert-mode
SMBus-two-wire-Interface
Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
Configure the below parameters :
Search (Ctrl+F)
Pinout view
System view
I2C1_SDA
I2C1_SCL
STM32L476RGTx
LQFP64

- Set the I2C **Mode** as **I2C** (standard mode) and the I2C Frequency at **100 KHz**


Pinout & Configuration
Clock Configuration
Software Packs
Categories
A->Z
System Core
Analog
Timers
Connectivity
CAN1
I2C1
I2C2
I2C3
IRTIM
LPUART1
QUADSPI
SDMMC1
SPI1
SPI2
SPI3
SWPMI1
UART4
UART5
USART1
USART2
USART3
USB_OTG_FS
I2C1 Mode and Configuration
Mode
Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)

I2C I2C

Timing configuration

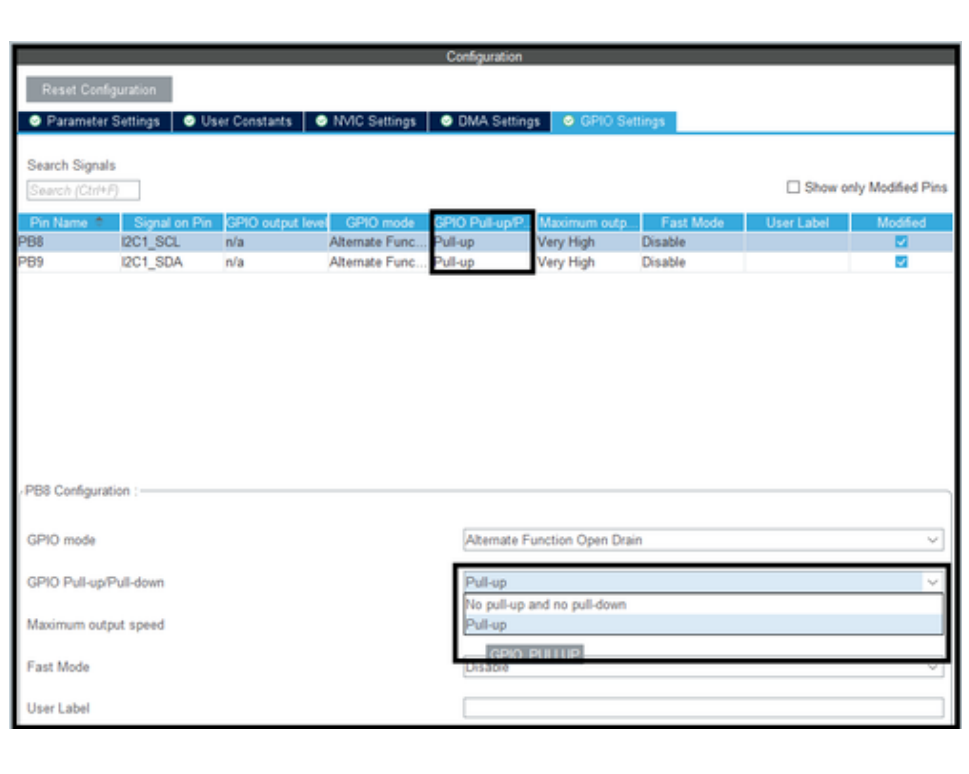
Custom Timing	Disabled
I2C Speed Mode	Standard Mode
I2C Speed Frequency (KHz)	100
Rise Time (ns)	0
Fall Time (ns)	0
Coefficient of Digital Filter	0
Analog Filter	Enabled
Timing	0x00000E14

Slave Features

Clock No Stretch Mode	Disabled
General Call Address Detection	Disabled
Primary Address Length selection	7-bit
Dual Address Acknowledged	Disabled
Primary slave address	0

2.3.2. GPIO Settings↑

- In the **GPIO Settings** tab
  - Set GPIO Pull-up/Pull-down to **Pull-up**
  - Set Maximum output speed to **Very High**

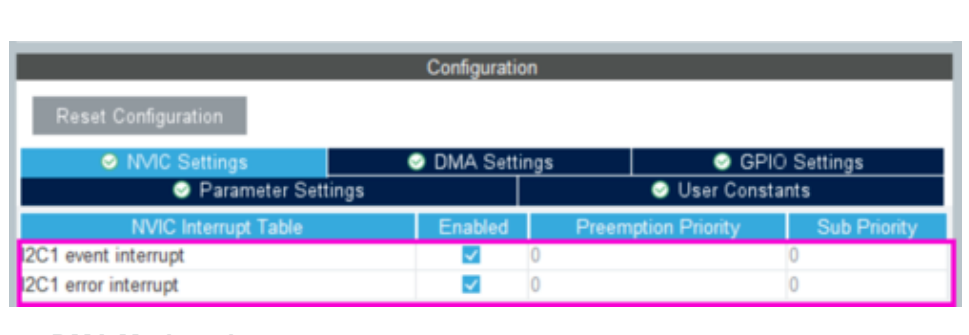

Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Search Signals
Search (Ctrl+F)
Show only Modified Pins
PB8 Configuration :
No pull-up and no pull-down
Pull-up
GPIO_PULLUP

Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/P...	Maximum outp...	Fast Mode	User Label	Modified
PB8	I2C1_SCL	n/a	Alternate Func...	Pull-up	Very High	Disable		☑
PB9	I2C1_SDA	n/a	Alternate Func...	Pull-up	Very High	Disable		☑

GPIO mode	Alternate Function Open Drain
GPIO Pull-up/Pull-down	Pull-up
Maximum output speed	
Fast Mode	Disable
User Label	

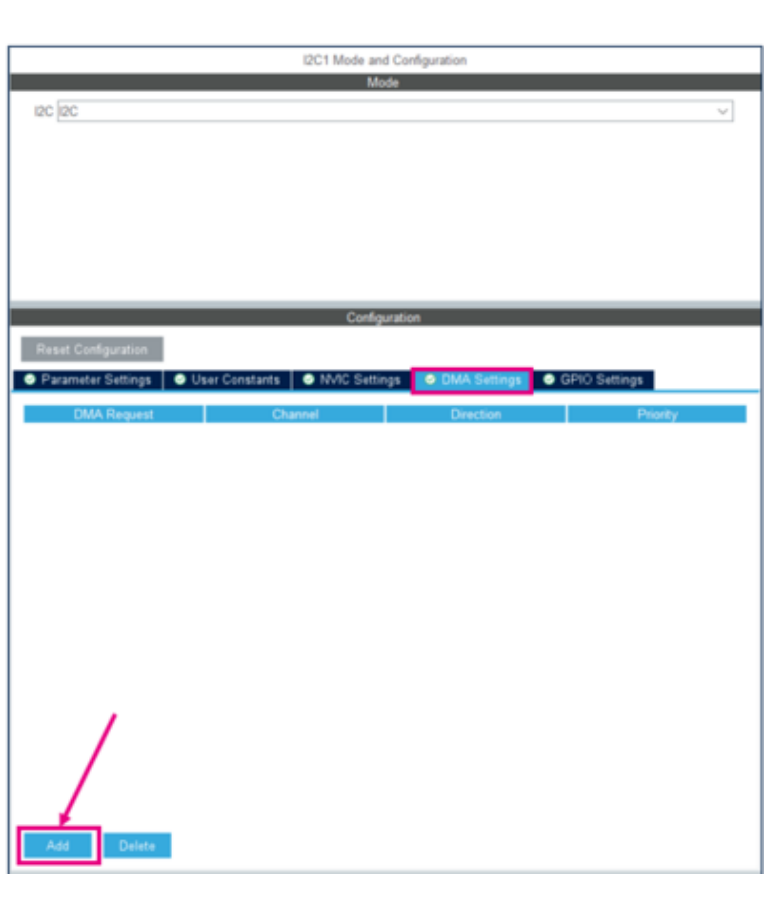
2.3.3. NVIC or DMA settings↑

- **Interrupt Mode only**
  - In the **NVIC settings** tab
  - Enable interrupt

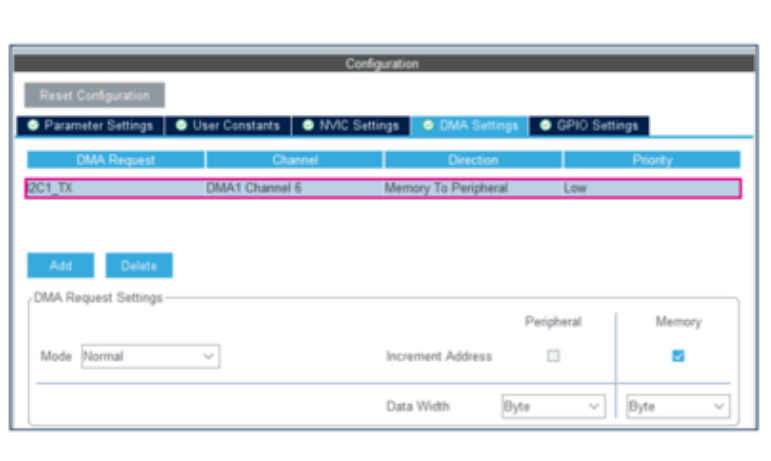

Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
I2C1 event interrupt	☑	0	0
I2C1 error interrupt	☑	0	0

- **DMA Mode only**
  - In the **DMA settings** tab


I2C1 Mode and Configuration
Mode
I2C I2C
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
DMA Request
Channel
Direction
Priority
Add
Delete

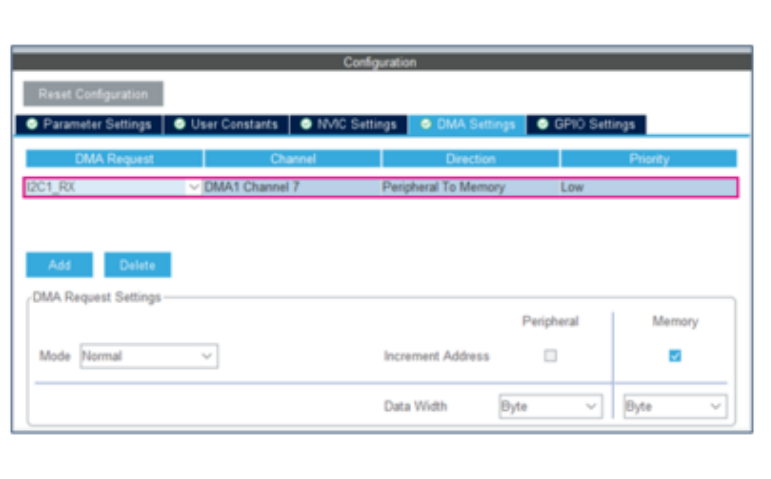
  - Add I2C1_TX for Master


Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Add
Delete
DMA Request Settings
Peripheral
Memory

DMA Request	Channel	Direction	Priority
I2C1_TX	DMA1 Channel 6	Memory To Peripheral	Low

Mode Normal
Increment Address: Peripheral ☐ Memory ☑
Data Width: Byte Byte

  - Add I2C1_RX for Slave

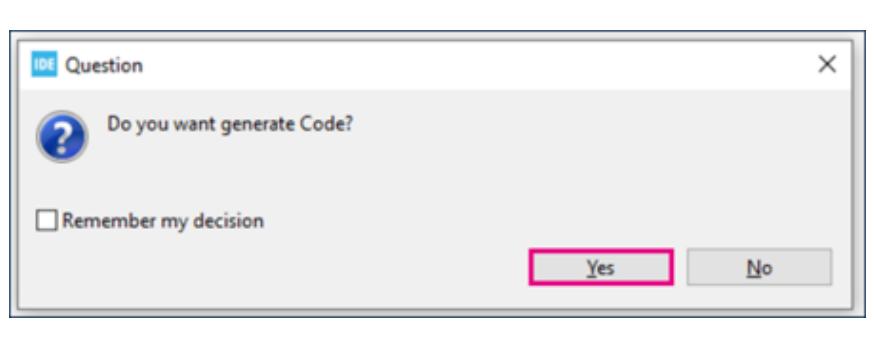

Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Add
Delete
DMA Request Settings
Peripheral
Memory

DMA Request	Channel	Direction	Priority
I2C1_RX	DMA1 Channel 7	Peripheral To Memory	Low

Mode Normal
Increment Address: Peripheral ☐ Memory ☑
Data Width: Byte Byte

2.4. Generate source code and edit main.c↑

- Click ***"Ctrl+S"*** to generate the project.


Question
Yes
No

Do you want generate Code?

☐ Remember my decision

2.4.1. On MASTER Board↑

- Open ***main.c*** in Project Explorer ***/ myproject / Src / main.c***.

Information
Insert your code between ***/* USER CODE BEGIN PV */*** and ***/* USER CODE END PV */*** tags

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t TX_Buffer [] = "A" ; // DATA to send
/* USER CODE END PV */
```

Information
Insert your code between ***/* USER CODE BEGIN 2 */*** and ***/* USER CODE END 2 */*** tags

- **Blocking Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Master_Transmit(&hi2c1,20,TX_Buffer,1,1000); //Sending in Blocking mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **Interrupt Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Master_Transmit_IT(&hi2c1,20,TX_Buffer,1); //Sending in Interrupt mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **DMA Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Master_Transmit_DMA(&hi2c1,20,TX_Buffer,1); //Sending in DMA mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

2.4.2. On SLAVE Board↑

- Open ***main.c*** in Project Explorer ***/ myproject / Src / main.c***.

Information
Insert your code between ***/* USER CODE BEGIN PV */*** and ***/* USER CODE END PV */*** tags

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t RX_Buffer [1] ; // DATA to receive
/* USER CODE END PV */
```

Information
Insert your code between ***/* USER CODE BEGIN 2 */*** and ***/* USER CODE END 2 */*** tags

- **Blocking Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Slave_Receive(&hi2c1 ,(uint8_t *)RX_Buffer, 1,1000); //Receiving in Blocking mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **Interrupt Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Slave_Receive_IT(&hi2c1 ,(uint8_t *)RX_Buffer, 1); //Receiving in Interrupt mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

- **DMA Mode**

```
/* USER CODE BEGIN 2 */
  HAL_I2C_Slave_Receive_DMA(&hi2c1 ,(uint8_t *)RX_Buffer, 1); //Receiving in DMA mode
  HAL_Delay(100);
/* USER CODE END 2 */
```

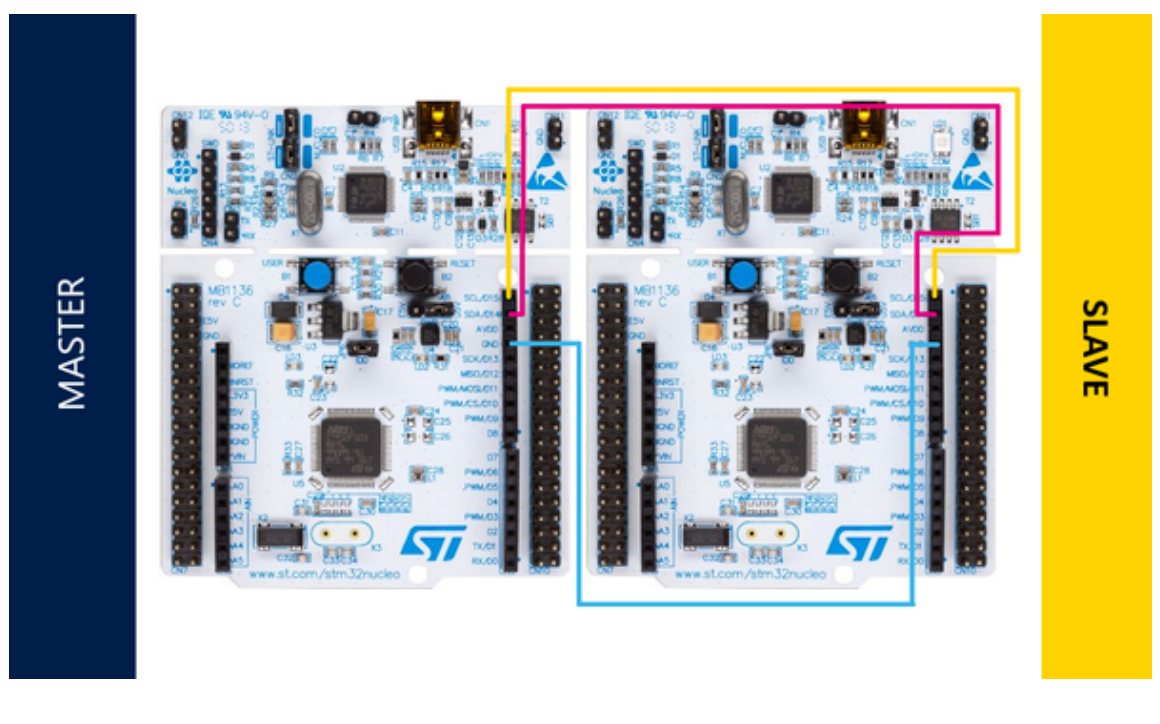
2.5. Compile and flash↑

- Click on ***Build*** button
- Click on ***Debug*** button (to run step by step)
- Or on ***Run*** button (to execute)

2.6. Project workspace↑

The following table shows the connection between 2 STM32 boards for using I2C bus. It requires only three wires.

NUCLEO-L476RG MASTER	NUCLEO-L476RG SLAVE	Pin description
PB8	PB8	SCL
PB9	PB9	SDA
GND	GND	Ground


MASTER
SLAVE

Getting started with STM32 system peripherals

All rights reserved (c) 2026 STMicroelectronics